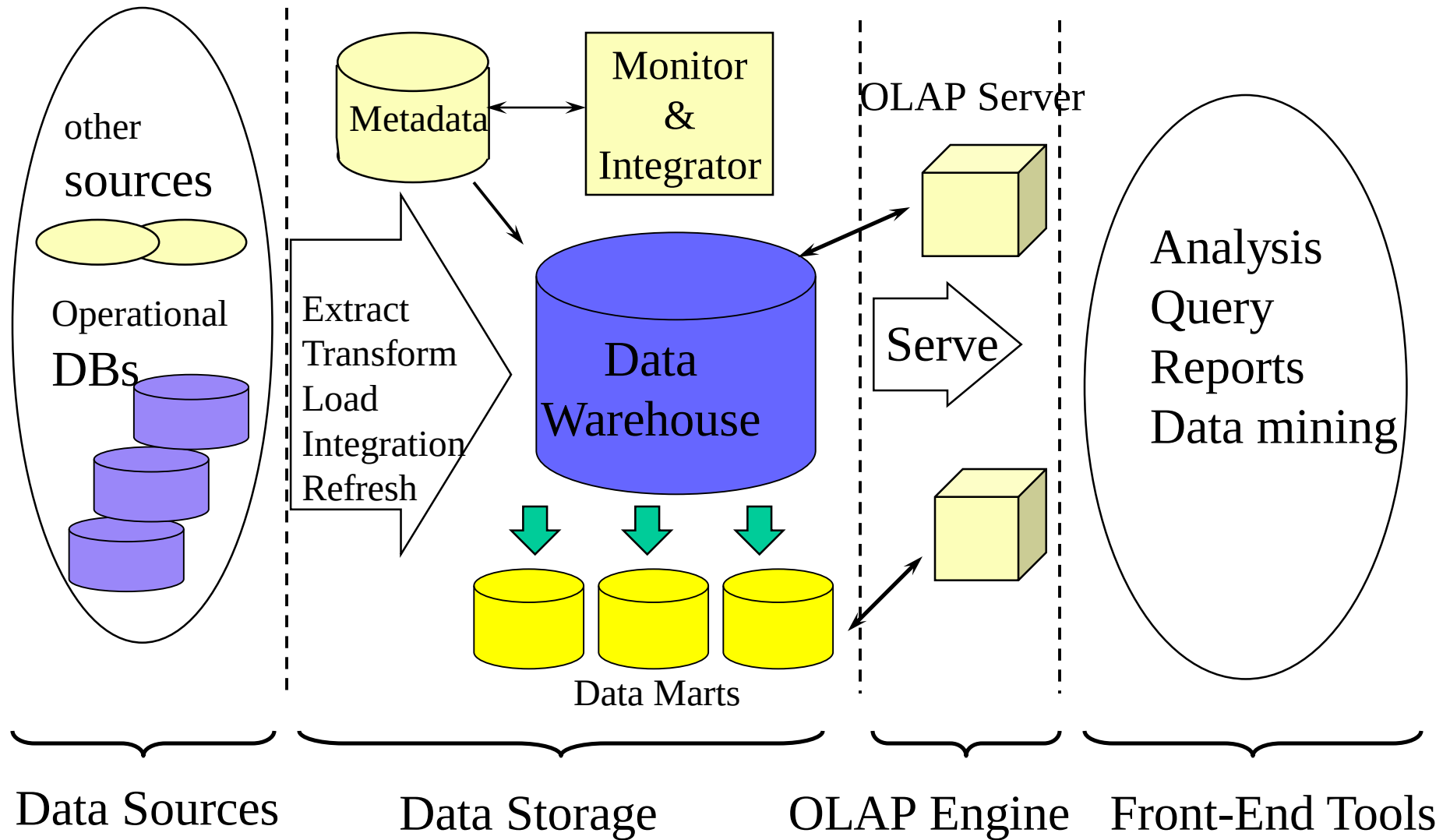


Multi-Tiered Architecture of data warehousing and data mining



What is XML?

- EXtensible Markup Language (XML)
- World Wide Web Consortium (W3C) recommendation Version 1.0 as of 10/02/1998.
- Describes data, rather than instructing a system on how to process it.
- Provides powerful capabilities for data integration and data-driven styling.
- Introduces new processing paradigms and requires new ways of thinking about Web development.
- A Meta-Markup Language, a set of rules for creating semantic tags used to describe data.

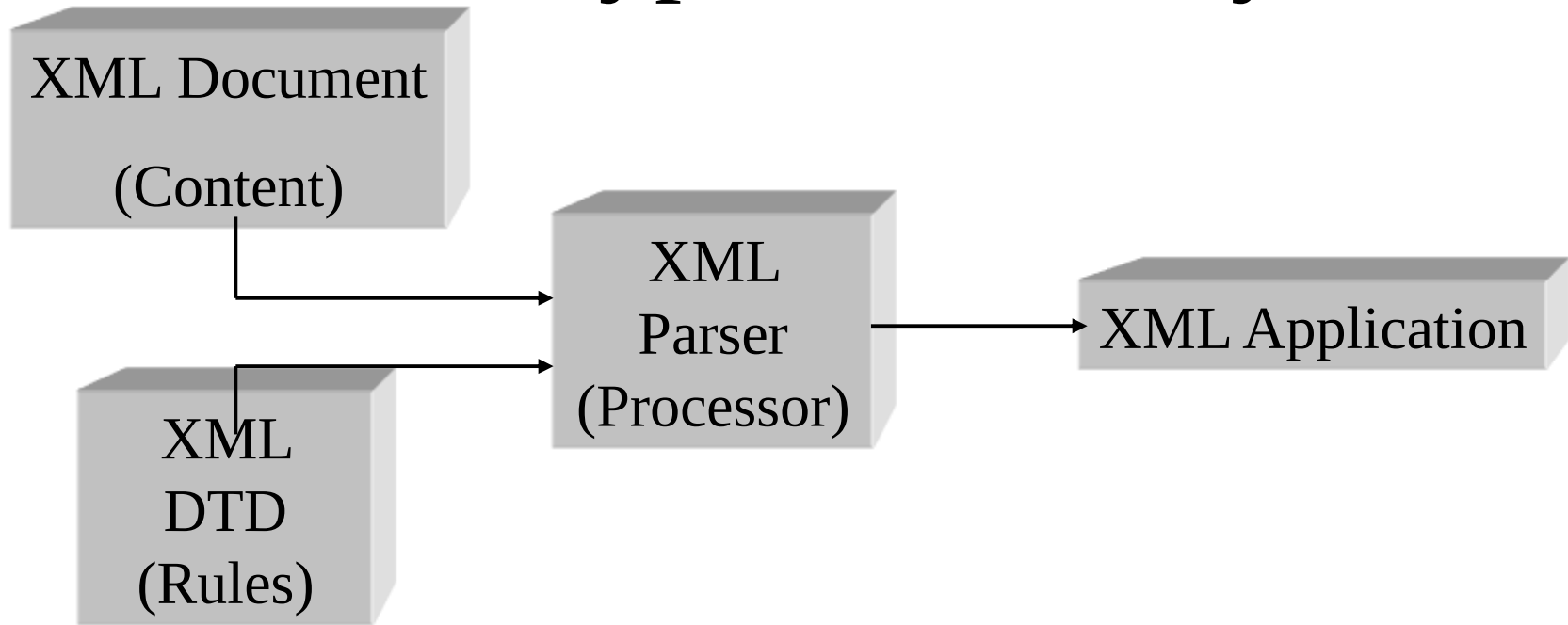
XML is Extensible

- The tags used to markup HTML documents and the structure of HTML documents are **predefined**.
- The author of HTML documents can only use tags that are defined **in the HTML standard**.
- XML allows the author to **define his own tags** and his own document structure.

Benefits of using XML

- It is structured.
- Documents are easily committed to a persistence layer.
- Platform independent, textual information.
- An open standard.
- Language independent.
- DOM and SAX are open, language-independent set of interfaces.
- It is Web enabled.

Typical XML System



- XML Document (content)
- XML Document Type Definition - DTD (structure definition; this is an operational part)
- XML Parser (conformity checker)
- XML Application (uses the output of the Parser to achieve your unique objectives)

How XML can be used?

- XML can keep data separated from your HTML document.
- XML can also store data inside HTML documents (Data Islands).
- XML can be used to exchange data.
- XML can be used to store data.

XML Syntax

- An example XML document.

```
<?xml version="1.0"?>
<note>
  <to>Tan Siew Teng</to>
  <from>Lee Sim Wee</from>
  <heading>Reminder</heading>
  <body>Don't forget the Golf Championship this
    weekend!</body>
</note>
```

Example (cont'd)

- The first line in the document: *The XML declaration* should always be included.
- It defines the XML version of the document.
- In this case the document conforms to the 1.0 specification of XML.

<?xml version="1.0"?>

- The next line defines the first element of the document (the root element):

<note>

Example (cont'd)

- The next lines defines 4 child elements of the root (to, from, heading, and body):

```
<to>Tan Siew Teng</to>  
<from>Lee Sim Wee</from>  
<heading>Reminder</heading>  
<body>Don't forget the Golf Championship this  
weekend!</body>
```

- The last line defines the end of the root element:

```
</note>
```

What is an XML element?

- An XML element is made up of a *start tag*, an *end tag*, and data in between.

`<Sport>Golf</Sport>`

- The name of the element is enclosed by the less than and greater than characters, and these are called tags.
- The start and end tags describe the data within the tags, which is considered the *value* of the element.
- For example, the following XML element is a `<player>` element with the value “Tiger Wood.”

`<player>Tiger Wood</player>`

There are 3 types of tags

- **Start-Tag**
 - In the example <Sport> is the start tag. It defines type of the element and possible *attribute specifications*
`<Player firstname="Wood" lastname="Tiger">`
- **End-Tag**
 - In the example </Sport> is the end tag. It identifies the type of element that tag is ending. Unlike start tag end tag cannot contain *attribute specifications*.
- **Empty Element Tag**
 - Like start tag this has *attribute specifications* but it does not need an end tag. Denotes that element is empty (does not have any other elements). Note the symbol for ending tag '/' before '>'
`<Player firstname="Wood" lastname="Tiger"/>`

XML elements must have a closing tag

In HTML some elements do not have to have a closing tag.

The following code is legal in HTML:

```
<p>This is a paragraph  
<p>This is another paragraph
```

In XML all elements must have a closing tag like this:

```
<p>This is a paragraph</p>  
<p>This is another paragraph</p>
```

Rules for Naming Elements

- XML names should start with a letter or the underscore character.
- Rest of the name can contain letters, digits, dots, underscores or hyphens.
- No spaces in names are allowed.
- Names cannot start with 'xml' which is a reserved word.

XML tags are case sensitive

- XML tags are case sensitive. The tag `<Message>` is different from the tag `<message>`.
- Opening and closing tags must therefore be written with the same case:

`<message>This is correct</message>`

`<Message>This is incorrect</message>`

All XML elements must be properly nested

In HTML some elements can be improperly nested within each other like this:

```
<b><i>This text is bold and italic</b></i>
```

In XML all elements must be properly nested within each other like this

```
<b><i>This text is bold and italic</i></b>
```

XML documents must have a root tag

- Documents must contain a single tag pair to define the root element.
- All other elements must be nested within the root element.
- All elements can have sub (children) elements.
- Sub elements must be in pairs and correctly nested within their parent element:

```
<root>  
  <child>  
    <subchild>  
      </subchild>  
    </child>  
  </root>
```


XML Attributes

- XML attributes are normally used to describe XML elements, or to provide additional information about elements.
- An element can optionally contain one or more *attributes*. An attribute is a name-value pair separated by an equal sign (=).
- Usually, or most common, attributes are used to provide information that is not a part of the content of the XML document.
- Often the attribute data is more important to the XML parser than to the reader.

XML Attributes (cont'd)

- Attributes are always contained within the start tag of an element. Here are some examples:

```
<Player firstname="Wood" lastname="Tiger" />
```

Player - Element Name

Firstname - Attribute Name

Wood - Attribute Value

- HTML examples:

```

```

```
<a href="demo.asp">
```

- XML examples:

```
<file type="gif">
```

```
<person id="3344">
```

Attribute values must always be quoted

- XML elements can have attributes in name/value pairs just like in HTML.
- An element can optionally contain one or more *attributes*.
- In XML the attribute value must always be quoted.
- An attribute is a name-value pair separated by an equal sign (=).

`<CITY ZIP="01085">Westfield</CITY>`

- `ZIP="01085"` is an attribute of the `<CITY>` element.

What is a Comment ?

- Comments are informational help for the reader.
- These are ignored by XML processors.
- They are enclosed within "<!--" and "-->" tags.
`<!-- This is a comment -->`

What is a Processing Instruction ?

- Processing Instructions provide a way to send instructions to computer programs or applications. They are enclosed within "<?" and "?>" tags.

```
<? xml:stylesheet type="text/xsl" href="styler.xsl" ?>
```

xml:stylesheet - Application name

type="text/xsl" href="styler.xsl" - Instructions to the application

What is a DTD ?

- Document Type Declaration (DTD) is a mechanism (set of rules) to describe the structure, syntax and vocabulary of XML documents.
- It is a modeling language for XML but it does not follow the same syntax as XML.

Document Type Definition (DTD)

- Define the legal building blocks of an XML document.
- Set of rules to define document structure with a list of legal elements.
- Declared inline in the XML document or as an external reference.
- All names are user defined.
- Derived from SGML.
- One DTD can be used for multiple documents.
- Has ASCII format.
- DOCTYPE keyword.

Element Declaration

- Following lines show the possible syntaxes for element declaration

`<!ELEMENT reports (employee*)>`

`<!ELEMENT employee (ss_number, first_name, middle_name, last_name, email, extension, birthdate, salary)>`

`<!ELEMENT email (#PCDATA)>`

`<!ELEMENT extension EMPTY>`

`#PCDATA` - Parsed Character Data, meaning that elements can contain text. This requirement means that no child elements may be present in the element within which `#PCDATA` is specified

`EMPTY` - Indicates that this is the leaf element, cannot contain any more children

Occurrence

- There are notations to specify the number of occurrences that a child element can occur within the parent element.
- These notations can appear at the end of each child element
 - + - Element can appear one or several times
 - * - Element can appear zero or several times
 - ? - Element can appear zero or one time
 - nothing** - Element can appear only once (also, it must appear once)

Separators

- , - Elements on the right and left of comma must appear in the same order.
- | - Only one of the elements on the left or right of this symbol must appear.

Attribute Declaration

Syntaxes for attribute declaration

<!ATTLIST customer ID CDATA #REQUIRED>

<!ATTLIST customer Preferred (true | false) "false">

Customer	-	Element name
ID	-	Attribute type ID uniquely identifies an element
IDREF	-	Attribute with type IDREF point to elements with an ID attribute
Preferred	-	Attribute names
(true false)	-	Possible attribute values
False	-	Default attribute value
CDATA	-	Character data
#REQUIRED	-	Attribute value must be provided
#IMPLIED	-	If no value is provided, application must use its own default
#FIXED	-	Attribute value must be the one that is provided in DTD
NMTOKEN	-	Name token consists of letters, digits, periods, underscores, hyphens and colon characters

Why use a DTD?

- XML provides an application independent way of sharing data.
- With a DTD, independent groups of people can agree to use a common DTD for interchanging data.
- Your application can use a standard DTD to verify that data that you receive from the outside world is valid.
- You can also use a DTD to verify your own data.

Well-formed Document

```
<?xml version="1.0"?>
<TITLE>
<title>A Well-formed Documents</title>
<first>
  This is a simple
  <bold>well-formed</bold>
  document.
</first>
<TITLE>
```

Source:

[L1.xml](#)

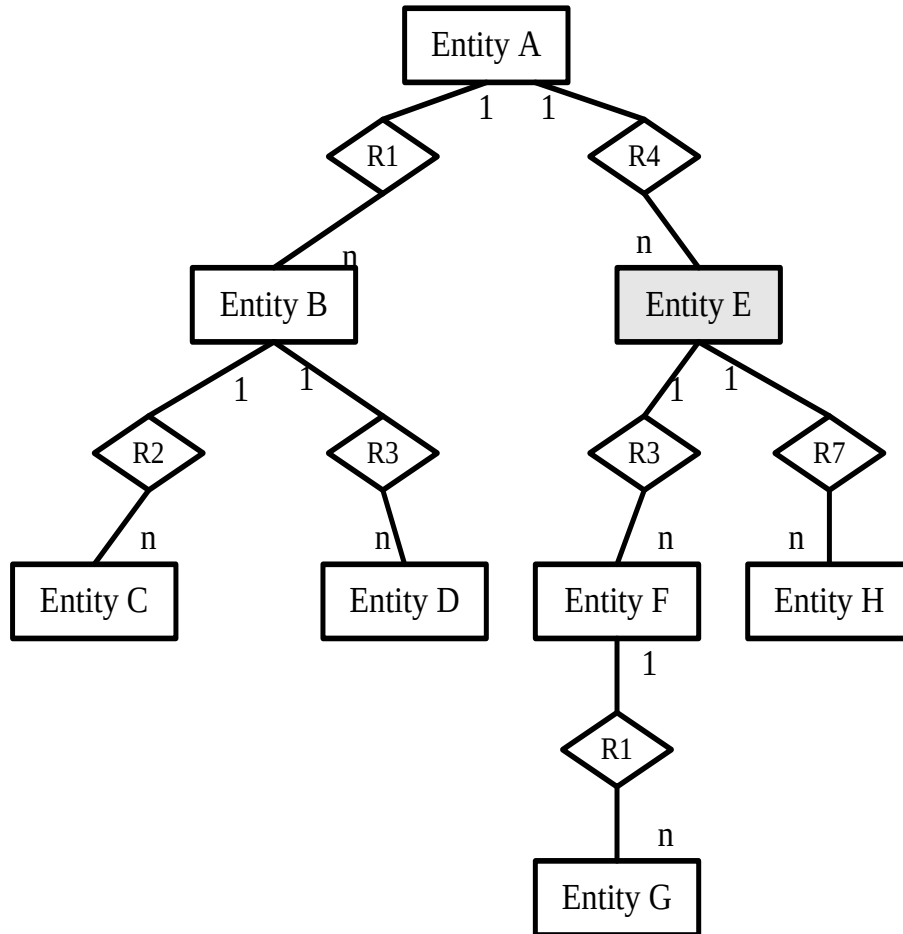
Rules for Well-formed Documents

- The first line of a well-formed XML document must be an XML declaration.
- All non-empty elements must have start tags and end tags with matching element names.
- All empty elements must end with />.
- All document must contain one root element.
- Nested elements must be completely nested within their higher-level elements.
- The only reserved entity references are &, ', > <, and ".

DTD Graph

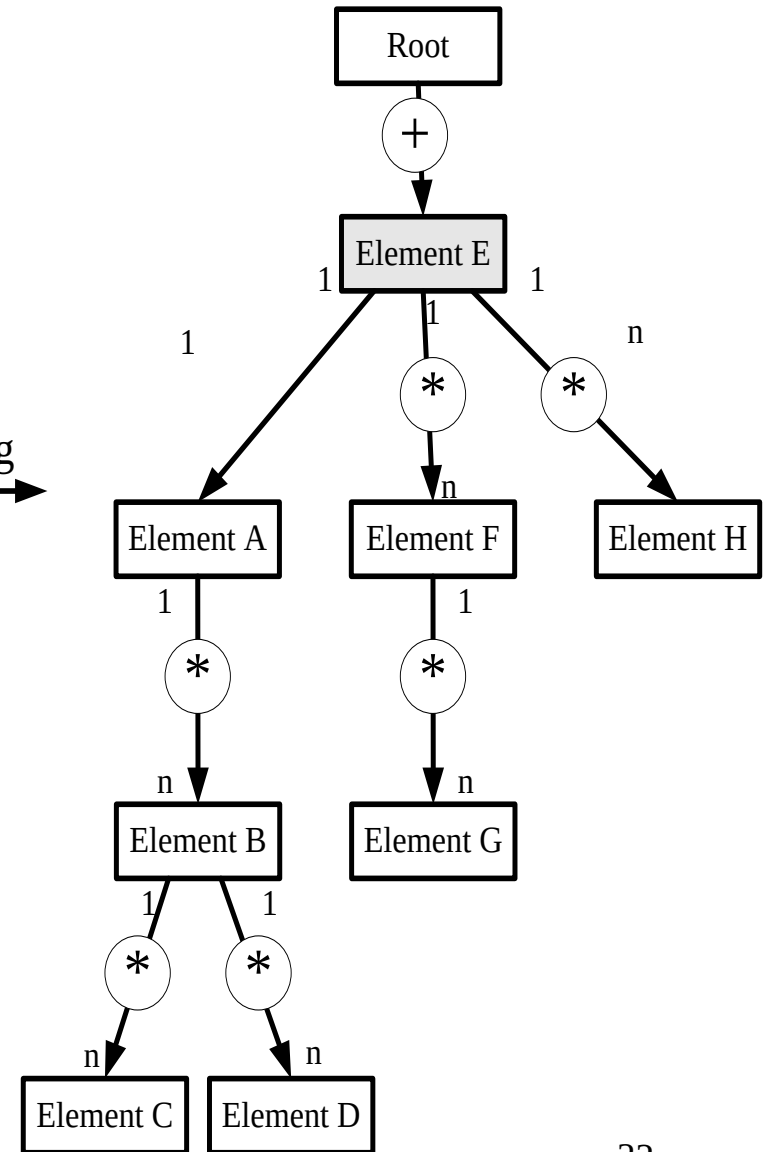
- Given the DTD information of the XML to be stored, we can create a structure called the Data Type Definition Graph that mirrors the structure of the DTD. Each node in the Data Type Definition graph represents an XML element in rectangle, an XML attribute in semi-cycle, and an operator in cycle. They are put together in a hierarchical containment under a root element node, with element nodes under a parent element node, separated by occurrence indicator in cycle.
- Facilities are available to link elements together with an Identifier (ID) and Identifier Reference (IDREF). An element with IDREF refers to an element with ID. Each ID must have a unique address. Nodes can refer to each other by using ID and IDREF such that nodes with IDREF referring to nodes with ID.

Extended Entity Relationship Model

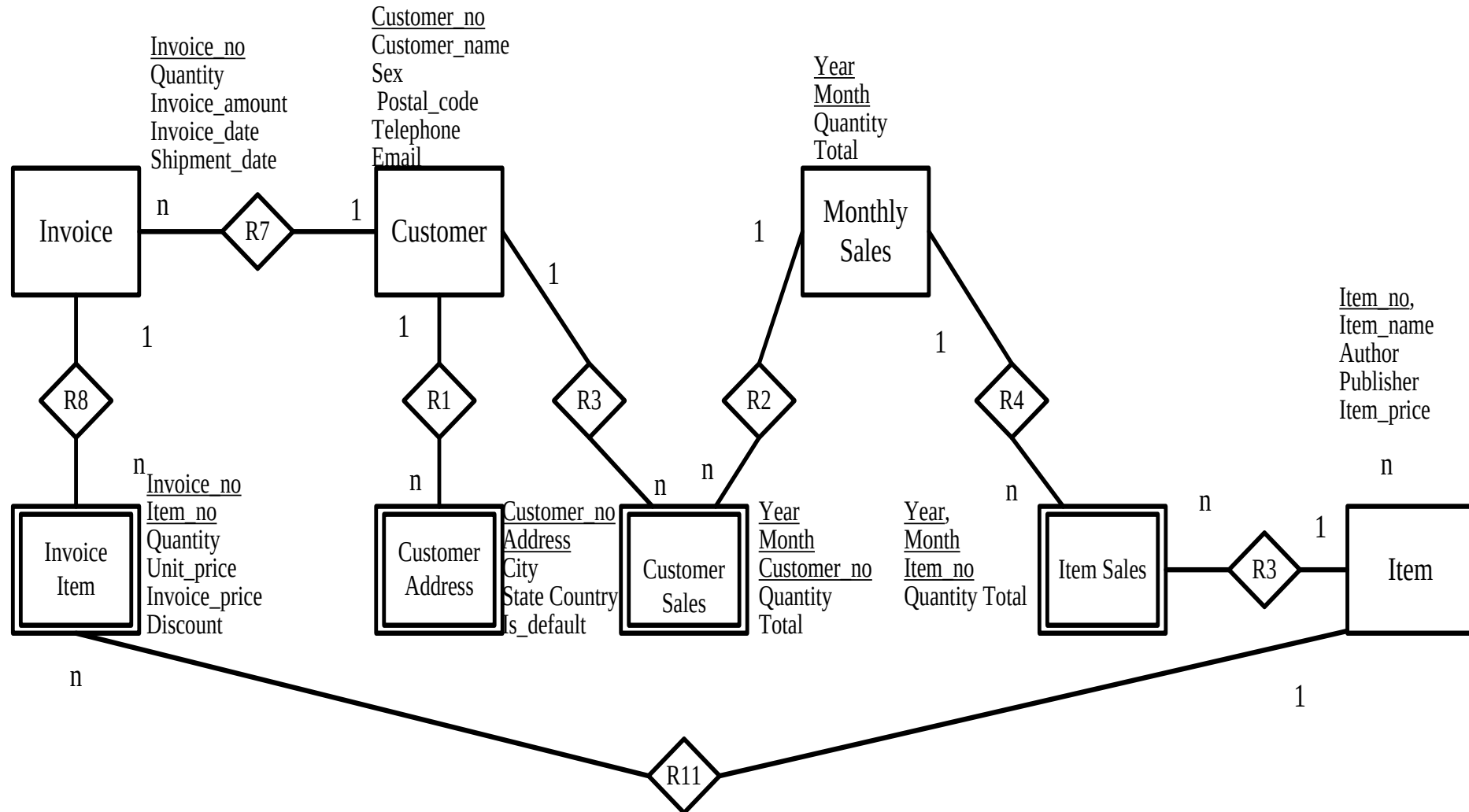


Mapping

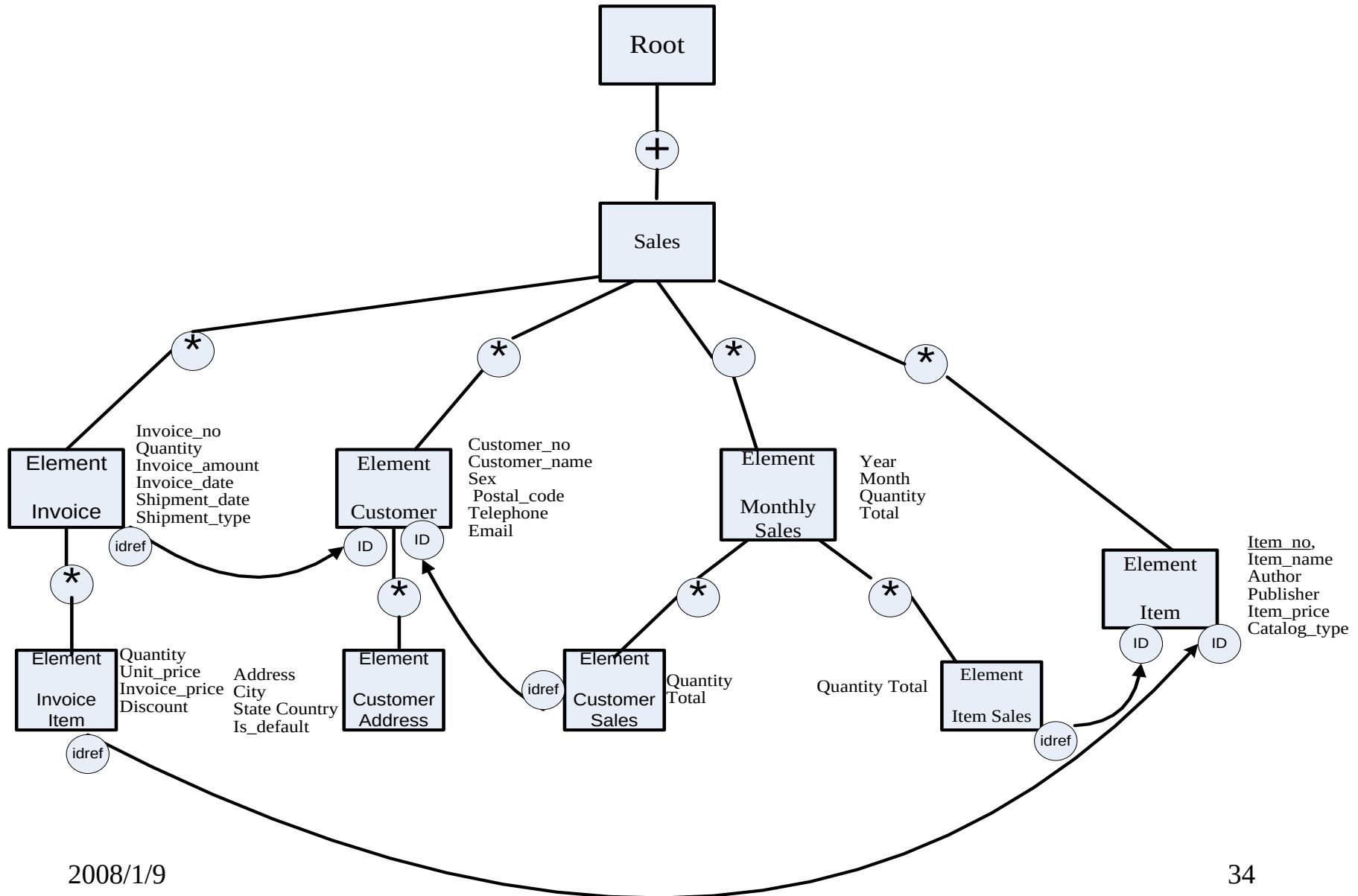
DTD Graph



An EER model for Customer Sales



A Sample DTD graph for Customer Sales



The mapped DTD from DTD Graph

```
<!ELEMENT Sales (Invoice*, Customer*, Item*, Monthly_sales*)>
<!ATTLIST Sales
  Status (New | Updated | History)          #required>
<!ELEMENT Invoice (Invoice_item*)>
<!ATTLIST Invoice
  Invoice_no          CDATA      #REQUIRED
  Quantity           CDATA      #REQUIRED
  Invoice_amount      CDATA      #REQUIRED
  Invoice_date        CDATA      #REQUIRED
  Shipment_date      CDATA      #IMPLIED
  Customer_idref      IDREF      #REQUIRED>
<!ELEMENT Customer (Customer_address*)>
<!ATTLIST Customer
  Customer_id         ID          #REQUIRED
  Customer_name       CDATA      #REQUIRED
  Customer_no         CDATA      #REQUIRED
  Sex                 CDATA      #IMPLIED
  Postal_code         CDATA      #IMPLIED
  Telephone           CDATA      #IMPLIED
  Email               CDATA      #IMPLIED>
<!ELEMENT Customer_address EMPTY>
<!ATTLIST Customer_address
  Address_type (Home|Office)          #REQUIRED
  Address      NMTOKENS               #REQUIRED
  City         CDATA      #IMPLIED
  State        CDATA      #IMPLIED
  Country      CDATA      #IMPLIED
  Customer_idref IDREF      #REQUIRED
  Is_default   (Y|N) "Y">
```

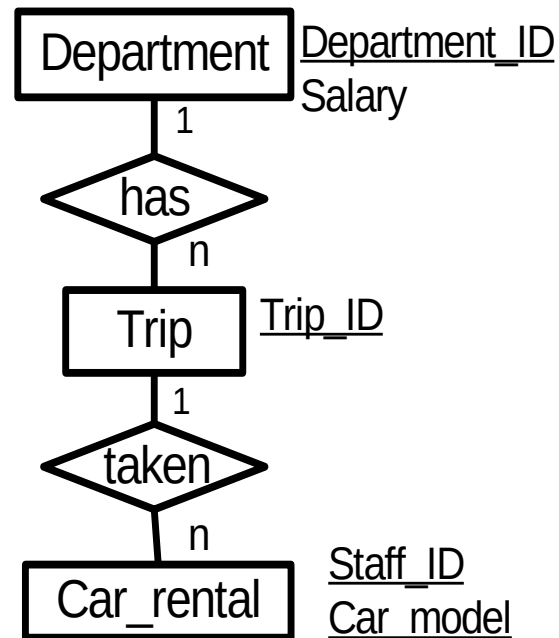
<!--ELEMENT Invoice_Item		EMPTY>
<!--ATTLIST Invoice_Item		
Quantity	CDATA	#REQUIRED
Unit_price	CDATA	#REQUIRED
Invoice_price	CDATA	#REQUIRED
Discount	CDATA	#REQUIRED
Item_idref	IDREF	REQUIRED>
<!--ELEMENT Item		EMPTY>
<!--ATTLIST Item		
Item_id	ID	#REQUIRED
Item_name	CDATA	#REQUIRED
Author	CDATA	#IMPLIED
Publisher	CDATA	#IMPLIED
Item_price	CDATA	#REQUIRED>
<!--ELEMENT Monthly_sales(Item_sales*, Customer_sales*)>		
<!--ATTLIST Monthly_sales		
Year	CDATA	#REQUIRED
Month	CDATA	#REQUIRED
Quantity	CDATA	#REQUIRED
Total	CDATA	#REQUIRED>
<!--ELEMENT Item_sales		EMPTY>
<!--ATTLIST Item_sales		
Quantity	CDATA	#REQUIRED
Total	CDATA	#REQUIRED
Item_idref	IDREF	#REQUIRED>
<!--ELEMENT Customer_sales		EMPTY>
<!--ATTLIST Customer_sales		
Quantity	CDATA	#REQUIRED
Total	CDATA	#REQUIRED
Customer_idref	IDREF	#REQUIRED>

Review Question 1

What are the similarity and dissimilarity between DTD and Well-formed Document?

Tutorial question 1

Map the following Extended Entity Relationship Model into an DTD Graph and a Document Type Definition (DTD)



An ER model for car rental

Reading assignment

Chapter 3 Schema Translation in “Information Systems Reengineering and Integration” Second Edition, by Joseph Fong, published by Springer, 2006, pp.142-154.